

Walchand College of Engineering, Sangli

Department of Computer Science and Engineering

Class: Final Year B.Tech(Computer Science and Engineering)

Year: 2025-26

Semester: 1

Course: High Performance Computing Lab

Practical No. 3

Exam Seat No: 22510019

Batch : B6

Title of practical:

Study and Implementation of schedule, nowait, reduction, ordered and collapse clauses

Problem Statement 1:

Analyse and implement a Parallel code for below program using OpenMP.

// C Program to find the minimum scalar product of two vectors (dot product)

Code snapshot :

```

5  int main() {
6      int i;
7      int n;
8
9      printf("Enter Size: ");
10     scanf("%d",&n);
11
12     int *a = malloc(n * sizeof(int));
13     int *b = malloc(n * sizeof(int));
14
15     long long dot = 0;
16
17     for (i = 0; i < n; i++) {
18         a[i] = i % 100;
19         b[i] = (n - i) % 100;
20     }
21
22     double start = omp_get_wtime();
23
24     #pragma omp parallel for reduction(+:dot_product)
25     for (i = 0; i < n; i++) {
26         dot += a[i] * b[i];
27     }
28
29     double end = omp_get_wtime();
30     printf("Minimum Scalar Product: %lld\n", dot);
31     printf("Time Taken: %f seconds\n", end - start);
32
33     free(a);
34     free(b);
35
36     return 0;
37 }

```

Screenshots:

```

soham@Vivobook16x-Soham: /mnt/d/SEM VII/HPCL/Assignment3$ ./ps1
Enter Size: 12
Minimum Scalar Product: 286
Time Taken: 0.004235 seconds

```

Information and analysis:

This program computes the scalar (dot) product of two integer vectors using OpenMP for parallelization. The vectors are initialized with sample values, and the dot product is calculated using a parallel for loop with reduction to efficiently sum the products across multiple threads.

For a vector size of **12**, the minimum scalar product was **286**, computed in **0.004235** seconds. While parallelization overhead is noticeable for small vectors, the approach scales well for larger data sizes, allowing significant speedup by utilizing multiple CPU cores.

Problem Statement 2:

Write OpenMP code for two 2D Matrix addition, vary the size of your matrices from 250, 500, 750, 1000, and 2000 and measure the runtime with one thread (Use functions in C to calculate the execution time or use GPROF)

- i. For each matrix size, change the number of threads from 2,4,8, and plot the speedup versus the number of threads.
- ii. Explain whether or not the scaling behaviour is as expected.

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment3$ ./ps2
Size: 250, Threads: 1, Time: 0.000387 seconds
Size: 250, Threads: 2, Time: 0.000606 seconds
Size: 250, Threads: 2, Speedup: 1.68
Size: 250, Threads: 4, Time: 0.000486 seconds
Size: 250, Threads: 4, Speedup: 1.81
Size: 250, Threads: 8, Time: 0.002496 seconds
Size: 250, Threads: 8, Speedup: 0.69

Size: 500, Threads: 1, Time: 0.001906 seconds
Size: 500, Threads: 2, Time: 0.001176 seconds
Size: 500, Threads: 2, Speedup: 2.75
Size: 500, Threads: 4, Time: 0.001274 seconds
Size: 500, Threads: 4, Speedup: 3.19
Size: 500, Threads: 8, Time: 0.002512 seconds
Size: 500, Threads: 8, Speedup: 4.11

Size: 750, Threads: 1, Time: 0.006260 seconds
Size: 750, Threads: 2, Time: 0.003154 seconds
Size: 750, Threads: 2, Speedup: 1.62
Size: 750, Threads: 4, Time: 0.002887 seconds
Size: 750, Threads: 4, Speedup: 1.63
Size: 750, Threads: 8, Time: 0.002632 seconds
Size: 750, Threads: 8, Speedup: 3.12

Size: 1000, Threads: 1, Time: 0.009541 seconds
Size: 1000, Threads: 2, Time: 0.004606 seconds
Size: 1000, Threads: 2, Speedup: 1.50
Size: 1000, Threads: 4, Time: 0.003709 seconds
Size: 1000, Threads: 4, Speedup: 1.09
Size: 1000, Threads: 8, Time: 0.003477 seconds
Size: 1000, Threads: 8, Speedup: 1.28

Size: 2000, Threads: 1, Time: 0.051064 seconds
Size: 2000, Threads: 2, Time: 0.023672 seconds
Size: 2000, Threads: 2, Speedup: 1.38
Size: 2000, Threads: 4, Time: 0.015665 seconds
Size: 2000, Threads: 4, Speedup: 1.42
Size: 2000, Threads: 8, Time: 0.015081 seconds
Size: 2000, Threads: 8, Speedup: 1.42
```

Information and analysis:

$$\text{Speedup} = T_{\text{serial}} / T_{\text{parallel}}$$

where

- T_{serial} = Time taken with **serially** (baseline)
- T_{parallel} = Time taken with **N threads**

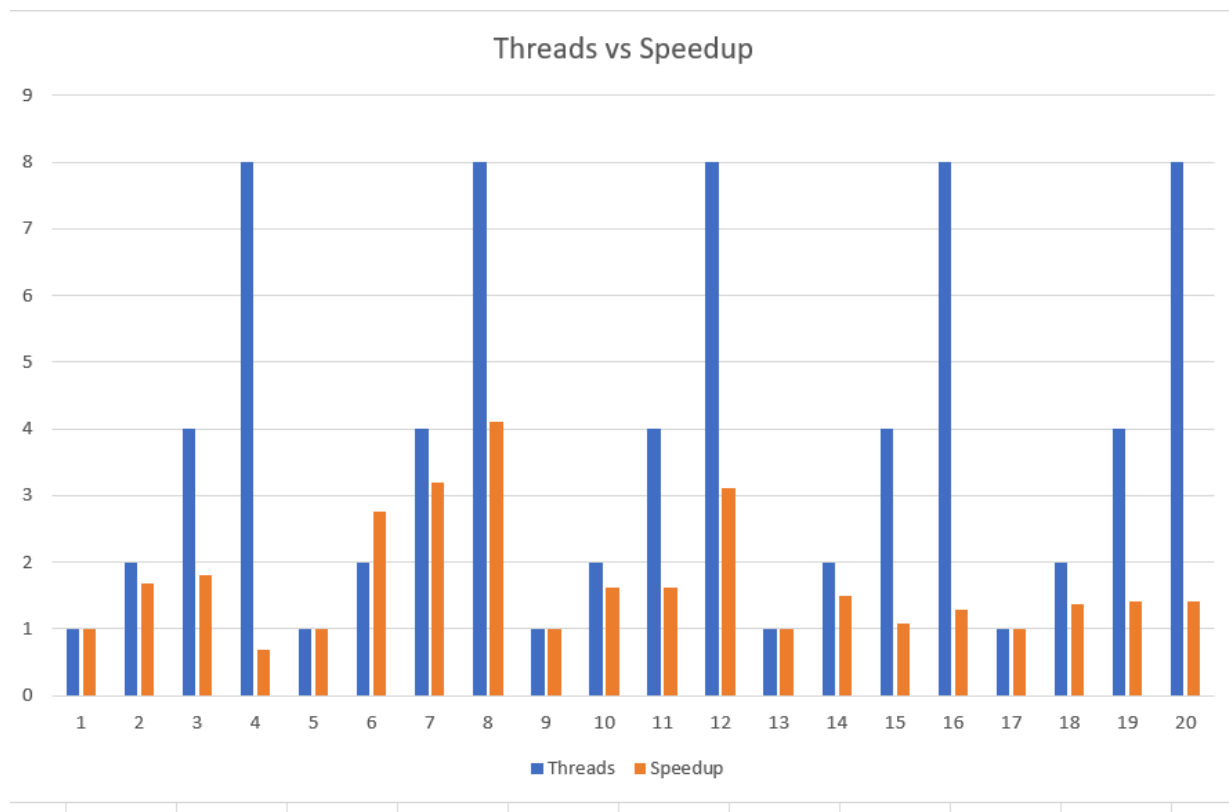
Small sized matrices (250, 500): Scaling is better, and noticeable speedup is observed.

Medium matrices (750): Speedup improves, but not linear.

Larger matrices (1000, 2000): Speedup stops increasing after a point, showing saturation.

- Speedup improves only up to a certain number of threads and then stabilizes or even decreases.

Plot :



Problem Statement 3:

For 1D Vector (size=200) and scalar addition, Write a OpenMP code with the following: i. Use STATIC schedule and set the loop iteration chunk size to various sizes when changing the size of your matrix. Analyze the speedup. ii. Use DYNAMIC schedule and set the loop iteration chunk size to various sizes when changing the size of your matrix. Analyze the speedup. iii. Demonstrate the use of nowait clause.

Screenshots:

```

STATIC, chunk=1, nowait=0, Time=0.007363
STATIC, chunk=1, nowait=1, Time=0.000134
DYNAMIC, chunk=1, nowait=0, Time=0.000232
DYNAMIC, chunk=1, nowait=1, Time=0.000194
STATIC, chunk=5, nowait=0, Time=0.000193
STATIC, chunk=5, nowait=1, Time=0.000191
DYNAMIC, chunk=5, nowait=0, Time=0.000192
DYNAMIC, chunk=5, nowait=1, Time=0.000196
STATIC, chunk=10, nowait=0, Time=0.000192
STATIC, chunk=10, nowait=1, Time=0.000186
DYNAMIC, chunk=10, nowait=0, Time=0.000193
DYNAMIC, chunk=10, nowait=1, Time=0.000166
STATIC, chunk=20, nowait=0, Time=0.000193
STATIC, chunk=20, nowait=1, Time=0.000190
DYNAMIC, chunk=20, nowait=0, Time=0.000356
DYNAMIC, chunk=20, nowait=1, Time=0.000192
STATIC, chunk=50, nowait=0, Time=0.000448
STATIC, chunk=50, nowait=1, Time=0.000140
DYNAMIC, chunk=50, nowait=0, Time=0.000279
DYNAMIC, chunk=50, nowait=1, Time=0.000110
STATIC, chunk=100, nowait=0, Time=0.000194
STATIC, chunk=100, nowait=1, Time=0.000196
DYNAMIC, chunk=100, nowait=0, Time=0.000199
DYNAMIC, chunk=100, nowait=1, Time=0.000194

```

- Very small chunks (like size = 1) caused overhead in static scheduling, making execution slower (0.0073s without nowait).
- Increasing the chunk size (5, 10, 20, ...) significantly reduced execution time, as the scheduling overhead decreased.
- For very large chunks (like 50 or 100), performance remained stable, though sometimes inconsistent due to load balancing differences.

Static vs. Dynamic Scheduling

- Static scheduling performed consistently better
- Dynamic scheduling was slightly slower at times due to runtime decisions of distributing chunks

Effect of nowait Clause

- Using `nowait` generally improved performance because it removed the unnecessary barrier synchronization at the end of the parallel loop.
- For example, with chunk size = 1, static scheduling improved from **0.007363s** → **0.000134s** just by adding `nowait`.

Github Link: [🌐 HPCL/Assignment3 at main · Somzee5/HPCL](#)

Final Year B.Tech: High Performance Computing Lab 2025-26 Sem I